DAA Practicals

Practical: Recursive and Non-Recursive Fibonacci Series

Aim: Write a program using recursive and non-recursive methods to calculate Fibonacci numbers and analyze their time and space complexity.

1. What is the Fibonacci Series?

The Fibonacci series is a sequence of numbers in which each number is the sum of the previous two numbers.

Example:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

The formula is:

\[F(n) = F(n-1) + F(n-2) \]

The first two numbers are:

- F(0) = 0
- F(1) = 1

2. What is Recursion?

Recursion means **a function calling itself** to solve a problem.

For example, to calculate Fibonacci(5), the function calls Fibonacci(4) and Fibonacci(3), and continues until it reaches the base condition.

Code Explanation

def fibonacci_recursive(n): if n <= 1: return n return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

- def defines a function.
- if n <= 1 is the base condition. It stops further function calls.
- return n returns 0 or 1.
- The last statement calls the same function again to calculate the previous two Fibonacci numbers and adds them.

Important: The base condition prevents infinite recursion.

3. What is Non-Recursive (Iterative)?

Non-recursive means solving a problem **without calling the same function again and again**.

It uses loops to calculate the result.

Code Explanation

def fibonacci_iterative(n): a = 0 b = 1 for i in range(n): print(a, end=" ") a, b = b, a + b print()

- a = 0 stores the first Fibonacci number.
- b = 1 stores the second Fibonacci number.
- for i in range(n) repeats the loop n times.
- print(a, end=" ") prints the current number.
- a, b = b, a + b updates the two numbers for the next iteration.
- print() moves the cursor to the next line.

5. Sample Input and Output

Input:

Enter the number of terms: 7

Output:

Recursive Fibonacci:

0 1 1 2 3 5 8

Non-Recursive Fibonacci:

0 1 1 2 3 5 8

Recursive Fibonacci:

Time Complexity: O(2^n)

Space Complexity: O(n)

Non-Recursive Fibonacci:

Time Complexity: O(n)

Space Complexity: O(1)

6. Time and Space Complexity

What is Time Complexity?

Time complexity tells us how much time an algorithm takes as the input size increases.

What is Space Complexity?

Space complexity tells us how much extra memory an algorithm uses.

Comparison Table

Method	Time Complexity	Space Complexity
Recursive	$O(2^n)$	O(n)
Non-Recursive	O(n)	O(1)

Recursive Complexity

- **Time: $O(2^n)$** – It makes many repeated function calls.
- **Space: O(n)** – It uses memory for function calls stored in the call stack.

Non-Recursive Complexity

- **Time: O(n)** – It uses a loop that runs n times.
- **Space: O(1)** – It uses only two variables, a and b, regardless of the input size.

The recursive method is slower for large inputs because it calculates the same values repeatedly. The iterative method is faster and uses less memory.

O(1)

O(log n)

O(n)

O(n log n)

$O(n^2)$

$O(2^n)$

102030405060501001502002Input size nGrowth

\(O(1):\;\text{1},\quad O(\log_2 n):\;\text{4}\)

\(O(n):\;\text{16},\quad O(n\log_2 n):\;\text{64}\)

\(O(n^2):\;\text{256},\quad O(2^n):\;\text{65,536}\)

View

\(\text{All curves}\)\(O(1)\)\(O(\log_2 n)\)\(O(n)\)\(O(n\log_2 n)\)\(O(n^2)\)\(O(2^n)\)

\(n\)

48163264

Give feedback

7. Important Viva Questions

Question	Answer
What is Fibonacci series?	A series where each number is the sum of the previous two numbers.
What is recursion?	A function calling itself.
What is iteration?	Repeating statements using loops.
What is a base condition?	A condition that stops recursive calls.
Which method is faster?	Non-recursive (iterative) method.
What is the time complexity of recursion?	$O(2^n)$.
What is the time complexity of iteration?	O(n).
What is a call stack?	Memory used to manage function calls.
Why is iteration better here?	It avoids repeated calculations and uses less memory.
Which loop is used in the iterative method?	For loop.

Practical: Huffman Encoding Using Greedy Strategy

1. Aim

To implement Huffman Encoding using a Greedy Algorithm to generate unique binary codes for characters based on their frequencies.

2. What is Huffman Encoding?

Huffman Encoding is a data compression technique used to reduce the size of data.

- Characters that appear more frequently get shorter binary codes.
- Characters that appear less frequently get longer binary codes.

- It helps save storage space.

Example:

- A character appearing frequently may get code 0.
- A character appearing less frequently may get code 1101.

3. What is a Greedy Strategy?

A Greedy Algorithm makes the best possible choice at each step.

In Huffman Encoding:

- It selects the two characters/nodes with the lowest frequencies.
- Combines them into a new node.
- Repeats this process until only one node remains.

This forms the **Huffman Tree**.

4. Explanation of the Code

A. Importing Library

import heapq

- heapq is a Python library.
- It creates a Min Heap.
- A Min Heap always gives the element with the smallest value first.

B. Creating a Node

class Node: def __init__(self, char, freq): self.char = char self.freq = freq self.left = None self.right = None

- A node stores a character and its frequency.
- left and right represent its child nodes.

C. Comparing Frequencies

def __lt__(self, other): return self.freq < other.freq

This compares two nodes based on their frequencies so that the Min Heap can select the smallest frequency first.

D. Huffman Encoding Function

def huffman_encoding(characters, frequencies): heap = []

This function generates Huffman codes. An empty heap is created to store the nodes.

E. Inserting Characters

for char, freq in zip(characters, frequencies): heapq.heappush(heap, Node(char, freq))

- zip() combines characters and their frequencies.
- heappush() inserts each node into the Min Heap.

F. Building the Huffman Tree

while len(heap) > 1: left = heapq.heappop(heap) right = heapq.heappop(heap) new_node = Node(None, left.freq + right.freq) new_node.left = left new_node.right = right heapq.heappush(heap, new_node)

This is the main part of the Greedy Algorithm.

1. Remove the two nodes with the lowest frequencies.
2. Add their frequencies.
3. Create a new parent node.
4. Insert the new node into the heap.
5. Repeat until only one node remains.

G. Generating Binary Codes

root = heap[0]codes = {}def generate_codes(node, code): if node is None: return if node.char is not None: codes[node.char] = code return generate_codes(node.left, code + "0") generate_codes(node.right, code + "1")

- The final node becomes the root of the Huffman Tree.
- Moving to the left adds 0 to the code.
- Moving to the right adds 1.
- When a character is reached, its binary code is stored.

H. Main Program

characters = ['A', 'B', 'C', 'D', 'E']frequencies = [50, 25, 15, 10, 5]codes = huffman_encoding(characters, frequencies)print("Huffman Codes:")for char in characters: print(char, ":", codes[char])

Here:

- characters stores the input characters.
- frequencies stores how many times each character appears.
- The function generates Huffman codes.

- Finally, the codes are printed.

5. Sample Input

Character	Frequency
A	50
B	25
C	15
D	10
E	5

6. Sample Output

Huffman Codes:

A : 0

B : 10

C : 110

D : 1110

E : 1111

Note: Huffman codes may differ depending on how equal-frequency nodes are selected, but they should still be valid.

7. Important Topics Covered

Topic	Meaning
Greedy Algorithm	Selects the best option at each step.
Huffman Encoding	Technique used for data compression.
Min Heap	Data structure that gives the smallest element first.
Huffman Tree	Binary tree used to generate codes.
Binary Code	Code made using 0 and 1.
Recursion	A function calling itself.
heapq	Python library used to implement a heap.

8. Time and Space Complexity

- **Time Complexity:** O(n log n), where n is the number of characters.
- **Space Complexity:** O(n), for storing the heap and Huffman tree.

9. Viva Questions and Answers

Q1. What is Huffman Encoding?
It is a data compression technique that assigns binary codes to characters based on their frequencies.

Q2. Which algorithm is used in Huffman Encoding?
Greedy Algorithm.

Q3. What is a Min Heap?
A data structure in which the smallest element is always available at the top.

Q4. Why are shorter codes assigned to frequent characters?
To reduce the total number of bits required to store data.

Q5. What is a Huffman Tree?
It is a binary tree used to generate Huffman codes.

Q6. What do 0 and 1 represent in the tree?
0 represents the left branch and 1 represents the right branch.

Q7. What is the use of heapq?
It helps select the two nodes with the lowest frequencies.

Q8. What is the main advantage of Huffman Encoding?
It reduces data size and saves storage space.

10. Conclusion

Huffman Encoding uses a Greedy Strategy to generate binary codes based on character frequencies. It assigns shorter codes to frequently occurring characters and longer codes to less frequent characters, helping in efficient data compression.

Practical: Fractional Knapsack Problem Using Greedy Method

1. Aim

To solve the Fractional Knapsack Problem using a Greedy Algorithm and find the maximum profit.

2. What is the Knapsack Problem?

The Knapsack Problem is an optimization problem in which we have:

- A bag (knapsack) with limited capacity.

- Different items with different weights and profits.
- We need to select items to get maximum profit without exceeding the bag's capacity.

Fractional Knapsack: We can take a complete item or a fraction (part) of an item.

For example, if an item weighs 20 kg, we can take only 10 kg of it if needed.

3. What is a Greedy Method?

A Greedy Algorithm selects the best option at each step to get the best result.

In Fractional Knapsack:

- Calculate the profit-to-weight ratio of every item.
- Select the item with the highest ratio first.
- Continue until the bag is full.

Formula:

Profit-to-Weight Ratio = Profit / Weight

4. Explanation of the Code

A. Taking Input Values

n = 3weights = [10, 20, 30]profits = [60, 100, 120]capacity = 50

- n = Total number of items.
- weights = Weight of each item.
- profits = Profit of each item.
- capacity = Maximum capacity of the bag.

B. Calculating Profit-to-Weight Ratio

items = []for i in range(n): ratio = profits[i] / weights[i] items.append((ratio, weights[i], profits[i]))

- Calculates the profit-to-weight ratio of each item.
- Stores the ratio, weight and profit together in a list.

Example:

For Item 1:

Ratio = 60 / 10 = 6

C. Sorting Items

items.sort(reverse=True)

- Sorts items according to their profit-to-weight ratio.
- reverse=True sorts them in descending order.
- The item with the highest ratio comes first.

D. Selecting Items

total_profit = 0for ratio, weight, profit in items: if capacity >= weight: capacity -= weight total_profit += profit

- Checks whether the complete item fits in the bag.
- If it fits, the complete item is selected.
- Its weight is subtracted from the remaining capacity.
- Its profit is added to the total profit.

E. Taking a Fraction of an Item

else: fraction = capacity / weight total_profit += profit * fraction capacity = 0 break

If the complete item does not fit:

- Calculate how much of the item can fit.
- Add the corresponding fraction of its profit.
- Set capacity to zero because the bag is full.
- break stops the loop.

F. Displaying the Result

print("Maximum Profit:", total_profit)

Displays the maximum profit obtained.

5. Sample Input

Item	Weight	Profit	Ratio
1	10	60	6
2	20	100	5
3	30	120	4

Knapsack Capacity = **50**

6. Working / Calculation

Items are selected according to the highest profit-to-weight ratio.

1. Select Item 1: Weight = 10, Profit = 60.
2. Select Item 2: Weight = 20, Profit = 100.
3. Remaining capacity = 50 - 10 - 20 = 20.
4. Item 3 has weight 30, but only 20 can fit.
5. Take 20/30 fraction of Item 3.

Profit from Item 3:

120 × (20/30) = 80

Total Profit = 60 + 100 + 80 = 240

7. Sample Output

Maximum Profit: 240.0

8. Important Topics Covered

Topic	**Meaning**
Greedy Algorithm	Selects the best option at each step.
Knapsack Problem	Selecting items within limited capacity.
Fractional Knapsack	Allows taking a part of an item.
Profit/Weight Ratio	Profit divided by weight.
Sorting	Arranging items in descending order of ratio.
Optimization	Finding the maximum possible profit.

9. Time and Space Complexity

- **Time Complexity: O(n log n)** – Because sorting the items takes O(n log n) time.
- **Space Complexity: O(n)** – Because we store item details in a separate list.

Here, n represents the total number of items.

10. Viva Questions and Answers

Q1. What is Fractional Knapsack?
It is a problem in which we select complete or partial items to get maximum profit.

Q2. Which method is used?
Greedy Method.

Q3. What is the profit-to-weight ratio?
It is the profit of an item divided by its weight.

Q4. Why do we sort items?
To select the item with the highest profit-to-weight ratio first.

Q5. Can we take a fraction of an item?
Yes, we can take a fraction of an item.

Q6. What is the difference between 0/1 and Fractional Knapsack?

- 0/1 Knapsack: We either take the complete item or leave it.
- Fractional Knapsack: We can take a part of an item.

Q7. What is the time complexity?
O(n log n).

Q8. What is the main advantage of the Greedy Method?
It is simple and efficient for finding the optimal solution to the Fractional Knapsack Problem.

11. Conclusion

The Fractional Knapsack Problem is solved using a Greedy Algorithm. Items are selected according to their profit-to-weight ratio. By selecting the highest-ratio items first and taking fractions when required, we obtain the maximum profit.

Remember for your practical: The most important steps are calculating the ratio, sorting items in descending order, selecting complete items, and taking a fraction when the remaining capacity is insufficient.

Sample Input

Enter number of items: 3

Enter profit: 60

Enter weight: 10

Enter profit: 100

Enter weight: 20

Enter profit: 120

Enter weight: 30

Enter knapsack capacity: 50

Practical: 0/1 Knapsack Problem Using Dynamic Programming

1. Aim

To solve the 0/1 Knapsack Problem using Dynamic Programming and find the maximum profit without exceeding the given capacity.

2. What is the Knapsack Problem?

The Knapsack Problem is an optimization problem in which:

- We have a bag (knapsack) with limited capacity.
- Each item has a weight and a profit.
- We need to select items to get maximum profit without exceeding the bag's capacity.

What is 0/1 Knapsack?

In 0/1 Knapsack, we have only two choices for each item:

- **1:** Select the complete item.
- **0:** Do not select the item.

We cannot take a fraction of an item.

Example: If an item weighs 10 kg, we must either take all 10 kg or leave it.

3. What is Dynamic Programming?

Dynamic Programming (DP) is a method of solving a big problem by dividing it into smaller problems.

- It stores the results of smaller problems in a table.
- It avoids solving the same problem again and again.
- It helps find the optimal solution efficiently.

In this practical, we use a **DP Table** to calculate the maximum profit.

4. Explanation of the Code

A. Taking Input

```
n = int(input("Enter number of items: "))


profit = []
```

```
weight = []


for i in range(n):
    p = int(input("Enter profit: "))
    w = int(input("Enter weight: "))


    profit.append(p)
    weight.append(w)
```

- n represents the total number of items.
- profit stores the profit of each item.
- weight stores the weight of each item.
- The for loop accepts profit and weight from the user.
- append() adds values to the lists.

B. Validating Weights

```
if any(w <= 0 for w in weight):
    print("Weight must be positive")
```

Checks whether any item has zero or negative weight. All weights must be positive.

C. Taking Knapsack Capacity

```
capacity = int(input("Enter knapsack capacity: "))
```

Accepts the maximum weight that the bag can carry.

D. Creating the DP Table

```
dp = [[0 for j in range(capacity + 1)]
      for i in range(n + 1)]
```

- Creates a two-dimensional table.
- Rows represent the number of items.
- Columns represent the knapsack capacity.
- Initially, all values are zero.

E. Filling the DP Table

```
for i in range(1, n + 1):
  for w in range(1, capacity + 1):

    if weight[i-1] <= w:
      dp[i][w] = max(
        profit[i-1] + dp[i-1][w - weight[i-1]],
        dp[i-1][w]
      )

    else:
      dp[i][w] = dp[i-1][w]
```

This is the main part of Dynamic Programming.

For every item, the program checks two possibilities:

1. **Include the item:** Add its profit to the best profit possible with the remaining capacity.
2. **Exclude the item:** Keep the previous maximum profit.

max() selects whichever option gives more profit.

If the item's weight is greater than the current capacity, we cannot select it.

F. Displaying the DP Table

```
print("DP Table:")

for row in dp:
  print(row)
```

Displays the completed DP table, which stores the maximum profit for different capacities and item counts.

G. Finding Maximum Profit

```
print("Maximum Profit:", dp[n][capacity])
```

The last cell of the DP table contains the maximum possible profit.

H. Finding Selected Items

```
w = capacity

for i in range(n, 0, -1):

  if dp[i][w] != dp[i-1][w]:
    print("Item", i)
    w = w - weight[i-1]
```

- Starts from the last cell of the DP table.
- Checks whether the current item was selected.
- If the value changes, the item is included.
- Subtracts its weight from the remaining capacity.

This process is called **backtracking**.

5. Sample Input

Enter number of items: 3

Enter profit: 60

Enter weight: 10

Enter profit: 100

Enter weight: 20

Enter profit: 120

Enter weight: 30

Enter knapsack capacity: 50

6. Working / Calculation

Given:

Item	Weight	Profit
1	10	60
2	20	100
3	30	120

Knapsack Capacity = **50**

The best selection is:

- Item 2: Weight = 20, Profit = 100
- Item 3: Weight = 30, Profit = 120

Total Weight = 20 + 30 = 50

Total Profit = 100 + 120 = **220**

7. Sample Output

Maximum Profit: 220

Selected Items:

Item 3

Item 2

The program also displays the complete DP table.

8. Important Topics Covered

Topic	Meaning
0/1 Knapsack	Select the complete item or leave it.
Dynamic Programming	Solves problems using smaller subproblems.
DP Table	Stores calculated maximum profits.
Optimization	Finding the maximum possible profit.
Backtracking	Finds which items were selected.
max()	Selects the greater value.
append()	Adds elements to a list.

9. Time and Space Complexity

- **Time Complexity: O(n × W)** – We check every item for every possible capacity.
- **Space Complexity: O(n × W)** – We store results in a two-dimensional DP table.

Here:

- n = Number of items.
- W = Knapsack capacity.

10. Viva Questions and Answers

Q1. What is 0/1 Knapsack?
It is a problem where each item is either selected completely or not selected.

Q2. Which technique is used?
Dynamic Programming.

Q3. What is Dynamic Programming?
It is a technique that solves smaller subproblems and stores their results to avoid repeated calculations.

Q4. What is a DP Table?
It is a table that stores the maximum profit for different items and capacities.

Q5. Can we take half an item in 0/1 Knapsack?
No, we must take the complete item or leave it.

Q6. What is backtracking?
It is the process of moving backwards through the DP table to find the selected items.

Q7. What is the time complexity?
O(n × W).

Q8. What is the difference between Fractional and 0/1 Knapsack?

- Fractional Knapsack allows taking part of an item.
- 0/1 Knapsack allows only complete selection or rejection.

Q9. Why do we use max()?
To choose the option that gives maximum profit.

11. Conclusion

The 0/1 Knapsack Problem is solved using Dynamic Programming. The algorithm creates a DP table, checks whether to include or exclude each item, and finds the maximum possible profit without exceeding the given capacity.

Practical: N-Queens Problem Using Backtracking

1. Aim

To solve the N-Queens Problem using Backtracking by placing the first Queen at a given position and arranging the remaining Queens safely on the chessboard.

2. What is the N-Queens Problem?

The N-Queens Problem is a puzzle in which we have to place **N Queens on an N × N chessboard** so that no two Queens attack each other.

Rules:

- No two Queens should be in the same row.
- No two Queens should be in the same column.
- No two Queens should be on the same diagonal.

In our practical, the first Queen is already placed by the user, and the program places the remaining Queens.

Example: For N = 4, we need to place 4 Queens on a 4 × 4 chessboard.

3. What is Backtracking?

Backtracking is an algorithmic technique that tries different possible solutions.

- First, it places a Queen in a safe position.
- Then it moves to the next row.
- If no safe position is available, it goes back and changes the previous Queen's position.
- This process continues until a valid solution is found.

In simple words, **Backtracking means trying, checking, and going back if something goes wrong.**

4. Explanation of the Code

A. Taking Input and Creating the Board

n = int(input("Enter number of Queens: "))board = [[0 for j in range(n)] for i in range(n)]

- n represents the number of Queens.
- Creates an N × N chessboard.
- Initially, all positions contain 0.

- 0 means an empty position.

B. Placing the First Queen

row = int(input("Enter first Queen row (0 to n-1): "))col = int(input("Enter first Queen column (0 to n-1): "))

Takes the row and column of the first Queen from the user.

if not (0 <= row < n and 0 <= col < n): print("Invalid position")else: board[row][col] = 1

- Checks whether the entered position is valid.
- 1 represents a Queen.
- Places the first Queen at the given position.

C. is_safe() Function

def is_safe(board, row, col, n):

This function checks whether a Queen can be safely placed at a particular position.

It checks three directions:

1. Upper Column

for i in range(row): if board[i][col] == 1: return False

Checks whether another Queen is present in the same column.

2. Upper Left Diagonal

i = row - 1j = col - 1while i >= 0 and j >= 0: if board[i][j] == 1: return False i -= 1 j -= 1

Checks whether another Queen is present in the upper-left diagonal.

3. Upper Right Diagonal

i = row - 1j = col + 1while i >= 0 and j < n: if board[i][j] == 1: return False i -= 1 j += 1

Checks whether another Queen is present in the upper-right diagonal.

If no Queen is found in these directions, the function returns True, meaning the position is safe.

D. solve() Function

def solve(board, row, n):

This is the main function that solves the N-Queens Problem using Backtracking.

1. Base Condition

if row == n: return True

If all rows have been successfully filled, the solution is complete.

2. Skip First Queen's Row

if 1 in board[row]: return solve(board, row + 1, n)

If a Queen is already placed in the current row, the program skips that row and moves to the next one.

3. Try Different Positions

for col in range(n): if is_safe(board, row, col, n): board[row][col] = 1

Checks every column in the current row. If a position is safe, it places a Queen there.

4. Recursive Call

if solve(board, row + 1, n): return True

Moves to the next row and tries to place another Queen.

5. Backtracking

board[row][col] = 0

If the selected position does not lead to a solution, the Queen is removed and another position is tried.

E. Displaying the Final Board

if solve(board, 0, n): print("Final N-Queens Matrix:") for row in board: print(row)

- Calls the solve() function.
- If a solution is found, it displays the final chessboard.
- Otherwise, it displays that no solution exists.

---

5. Sample Input

Enter number of Queens: 4

Enter first Queen row (0 to n-1): 0

Enter first Queen column (0 to n-1): 1

6. Sample Output

Final N-Queens Matrix:

[0, 1, 0, 0]

[0, 0, 0, 1]

[1, 0, 0, 0]

[0, 0, 1, 0]

7. Important Topics Covered

Topic	Meaning
N-Queens Problem	Placing N Queens safely on an N × N chessboard.
Backtracking	Trying a solution and going back if it fails.
Recursion	A function calling itself.
Chessboard Matrix	A 2D list representing the chessboard.
Safe Position	A position where no other Queen can attack.
Diagonal Checking	Checking upper-left and upper-right directions.
Base Condition	Stops recursion when all Queens are placed.

8. Time and Space Complexity

- **Time Complexity:** O(N!), approximately, for the backtracking search.
- **Space Complexity:** $O(N^2)$ for the chessboard matrix.

Here, N represents the number of Queens.

9. Viva Questions and Answers

Q1. What is the N-Queens Problem?
It is a problem of placing N Queens on an N × N chessboard so that no two Queens attack each other.

Q2. Which technique is used?
Backtracking.

Q3. What is Backtracking?
It is a technique that tries different solutions and goes back when a solution fails.

Q4. What does 1 represent in the matrix?
It represents the position of a Queen.

Q5. What does 0 represent?
It represents an empty position.

Q6. What is the use of the is_safe() function?
It checks whether a position is safe for placing a Queen.

Q7. Why do we check diagonals?
Because a Queen can attack other Queens diagonally.

Q8. What is recursion?
When a function calls itself, it is called recursion.

Q9. What is the base condition?
When row == n, all Queens are placed, and the function returns True.

Q10. What happens during backtracking?
The previously placed Queen is removed, and another position is tried.

10. Conclusion

The N-Queens Problem is solved using Backtracking. The program checks safe positions and places Queens one by one. If a placement fails, it goes back and tries another position until a valid solution is found.

Practical: Analysis of Quick Sort Using Deterministic and Randomized Variants

1. Aim

To implement and analyze Quick Sort using Deterministic and Randomized methods and compare their execution times.

2. What is Quick Sort?

Quick Sort is a sorting algorithm that arranges elements in ascending or descending order.

It uses the **Divide and Conquer** technique.

It works in three steps:

1. Select a pivot element.
2. Divide the array into two parts:
    - Elements smaller than or equal to the pivot.
    - Elements greater than the pivot.
3. Apply Quick Sort recursively to both parts.

3. What is a Pivot?

A pivot is an element selected from the array to divide it into smaller parts.

For example:

Array: [40, 20, 60, 10, 50]

If pivot = 50:

- Left side: [40, 20, 10]
- Pivot: 50
- Right side: [60]

After sorting, the result is:

[10, 20, 40, 50, 60]

4. Types of Quick Sort

A. Deterministic Quick Sort

- Always selects the same position as the pivot.
- In our program, the last element is selected as the pivot.
- Its performance depends on the input arrangement.

B. Randomized Quick Sort

- Selects a random element as the pivot.
- The pivot changes randomly during execution.
- It reduces the chances of getting the worst-case performance repeatedly.

Main Difference:

Deterministic Quick Sort	**Randomized Quick Sort**
Selects a fixed pivot.	Selects a random pivot.
In our code, the last element is selected.	Uses random.choice().
Can perform poorly for certain inputs.	Reduces the chance of poor partitioning.

5. Explanation of the Code

A. Importing Libraries

import randomimport time

- random: Used to select a random pivot.
- time: Used to calculate the execution time.

B. Taking Input

n = int(input("Enter number of elements: "))arr = list(map(int, input("Enter array elements separated by spaces: ").split()))

- n stores the number of elements.
- arr stores the array entered by the user.
- split() separates the input values.
- map(int, ...) converts them into integers.

C. Validating Input

if n <= 0 or len(arr) != n: raise ValueError("Please enter a valid number of elements.")

Checks whether the number of elements entered matches the given size.

D. Deterministic Quick Sort

def deterministic_quick_sort(arr): if len(arr) <= 1: return arr pivot = arr[-1] left = [] right = [] for i in arr[:-1]: if i <= pivot: left.append(i) else: right.append(i) return deterministic_quick_sort(left) + [pivot] + deterministic_quick_sort(right)

Explanation:

- def defines the function.
- if len(arr) <= 1 checks the base condition.
- arr[-1] selects the last element as the pivot.
- left stores smaller elements.
- right stores greater elements.
- The function calls itself to sort both parts.
- Finally, it combines the sorted parts and pivot.

E. Randomized Quick Sort

def randomized_quick_sort(arr): if len(arr) <= 1: return arr pivot = random.choice(arr) left = [] middle = [] right = []

- Selects a random pivot using random.choice().
- left stores smaller elements.
- middle stores elements equal to the pivot.
- right stores greater elements.

for i in arr: if i < pivot: left.append(i) elif i == pivot: middle.append(i) else: right.append(i)

Divides the array into three parts based on the pivot.

return (randomized_quick_sort(left) + middle + randomized_quick_sort(right))

Recursively sorts the left and right parts and combines them with the middle part.

F. Measuring Execution Time

start = time.perf_counter()result1 = deterministic_quick_sort(arr.copy())time1 = time.perf_counter() - start

- perf_counter() records the starting time.
- Executes Deterministic Quick Sort.
- Calculates the total execution time.

The same process is repeated for Randomized Quick Sort.

G. Displaying Results

print("\nDeterministic Quick Sort:", result1)print("Execution Time:", time1)print("\nRandomized Quick Sort:", result2)print("Execution Time:", time2)

Displays the sorted arrays and their execution times.

6. Sample Input

Enter number of elements: 6

Enter array elements separated by spaces: 45 12 78 23 9 56

7. Sample Output

Original Array: [45, 12, 78, 23, 9, 56]

Deterministic Quick Sort: [9, 12, 23, 45, 56, 78]

Execution Time: 0.0000...

Randomized Quick Sort: [9, 12, 23, 45, 56, 78]

Execution Time: 0.0000...

Note: Execution time will vary depending on the computer and input.

8. Important Topics Covered

Topic	Meaning
Quick Sort	Sorting algorithm used to arrange elements.
Divide and Conquer	Divides a big problem into smaller problems.
Pivot	Element used to divide the array.
Deterministic	Uses a fixed pivot selection method.
Randomized	Selects a random pivot.
Recursion	A function calling itself.
Partitioning	Dividing elements around the pivot.
Execution Time	Time taken by an algorithm to execute.

9. Time and Space Complexity

Time Complexity

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

- Best case: Pivot divides the array into nearly equal parts.
- Average case: Array is divided reasonably well.
- Worst case: Pivot repeatedly creates highly unequal parts.

Space Complexity

In this particular Python implementation:

- **O(n)** auxiliary space for the temporary lists.
- Recursive calls also use stack space.

10. Viva Questions and Answers

Q1. What is Quick Sort?
Quick Sort is a sorting algorithm that uses the Divide and Conquer technique.

Q2. Which technique is used in Quick Sort?
Divide and Conquer.

Q3. What is a pivot?
A pivot is an element used to divide the array into smaller parts.

Q4. What is Deterministic Quick Sort?
It selects a pivot using a fixed rule, such as choosing the last element.

Q5. What is Randomized Quick Sort?
It selects the pivot randomly from the array.

Q6. What is the advantage of Randomized Quick Sort?
It reduces the chances of getting the worst-case performance repeatedly.

Q7. What is recursion?
When a function calls itself, it is called recursion.

Q8. What is the worst-case time complexity?
$O(n^2)$.

Q9. Which Python function measures execution time?
time.perf_counter().

Q10. Why do both algorithms give the same output?
Because both algorithms sort the same input array, but they select pivots differently.

11. Conclusion

Quick Sort is implemented using Deterministic and Randomized approaches. Both methods sort the array correctly, but they differ in pivot selection. Execution time is measured to analyze and compare their performance.